

Practical: 1

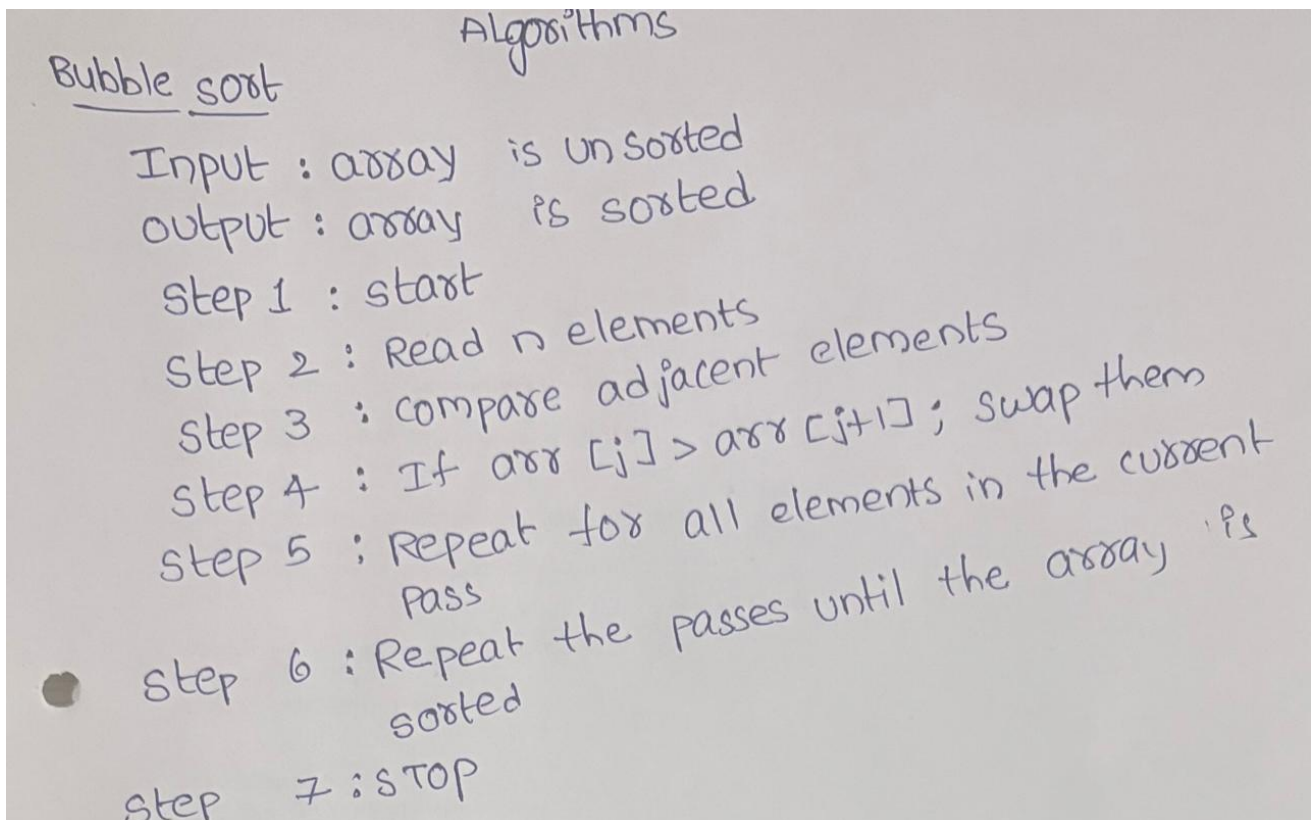
Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

29/07/2026

Aim: To Implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

1. Bubble sort

Algorithm:



Program:

```
#include <iostream>

using namespace std;

class bubblesort
{
public:
    int size;
    int arr[100];
    int i;

    void arrayinput()
    {
        cout << "enter the size of array" << endl;
        cin >> size;
        cout << "enter the array elements" << endl;
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
    }
}
```

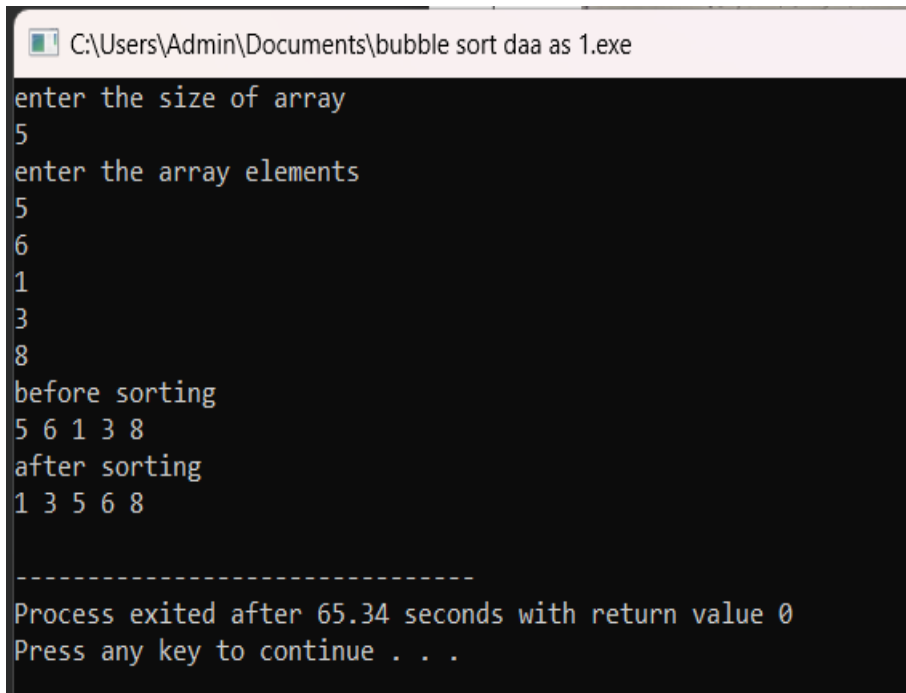
```
        cout << endl;
    }

    void bubblelogic()
    {
        int i, j, temp;
        for (i = 0; i < size - 1; i++)
        {
            for (j = 0; j < size - 1; j++)
            {
                if (arr[j] > arr[j + 1])
                {
                    temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
};

int main()
{
    bubblesort bs;
    bs.arrayinput();
    cout << "before sorting" << endl;
    bs.printarray();
    cout << "after sorting" << endl;
    bs.bubblelogic();
}
```

```
bs.printarray();  
return 0;  
}
```

Output:



```
C:\Users\Admin\Documents\bubble sort daa as 1.exe  
enter the size of array  
5  
enter the array elements  
5  
6  
1  
3  
8  
before sorting  
5 6 1 3 8  
after sorting  
1 3 5 6 8  
  
-----  
Process exited after 65.34 seconds with return value 0  
Press any key to continue . . .
```

time Complexity:

TIME complexity

Bubble sort

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n - i; j++) {
        if (arr[j] > arr[j + 1]) {
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
    
```

$n-1$
 $\frac{n(n-1)}{2}$

Worst case
The array is in unsorted

$$T(n) = n - 1 + \frac{n(n-1)}{2}$$

$$= n - 1 + \frac{n^2 - n}{2} \text{ Ignoring constant}$$

$$= \cancel{n} + \frac{n^2}{2} - \cancel{n}$$

$$\boxed{T(n) = O(n^2)} \rightarrow \text{Worst case}$$

Average case
The array is in random

$$T(n) = n - 1 + \frac{n(n-1)}{2}$$

$$= n - 1 + \frac{n^2 - n}{2} \text{ Ignoring constant}$$

$$= \cancel{n} + \frac{n^2}{2} - \cancel{n}$$

$$\boxed{T(n) = O(n^2)} \rightarrow \text{Average case}$$

Best case
The array is already sorted

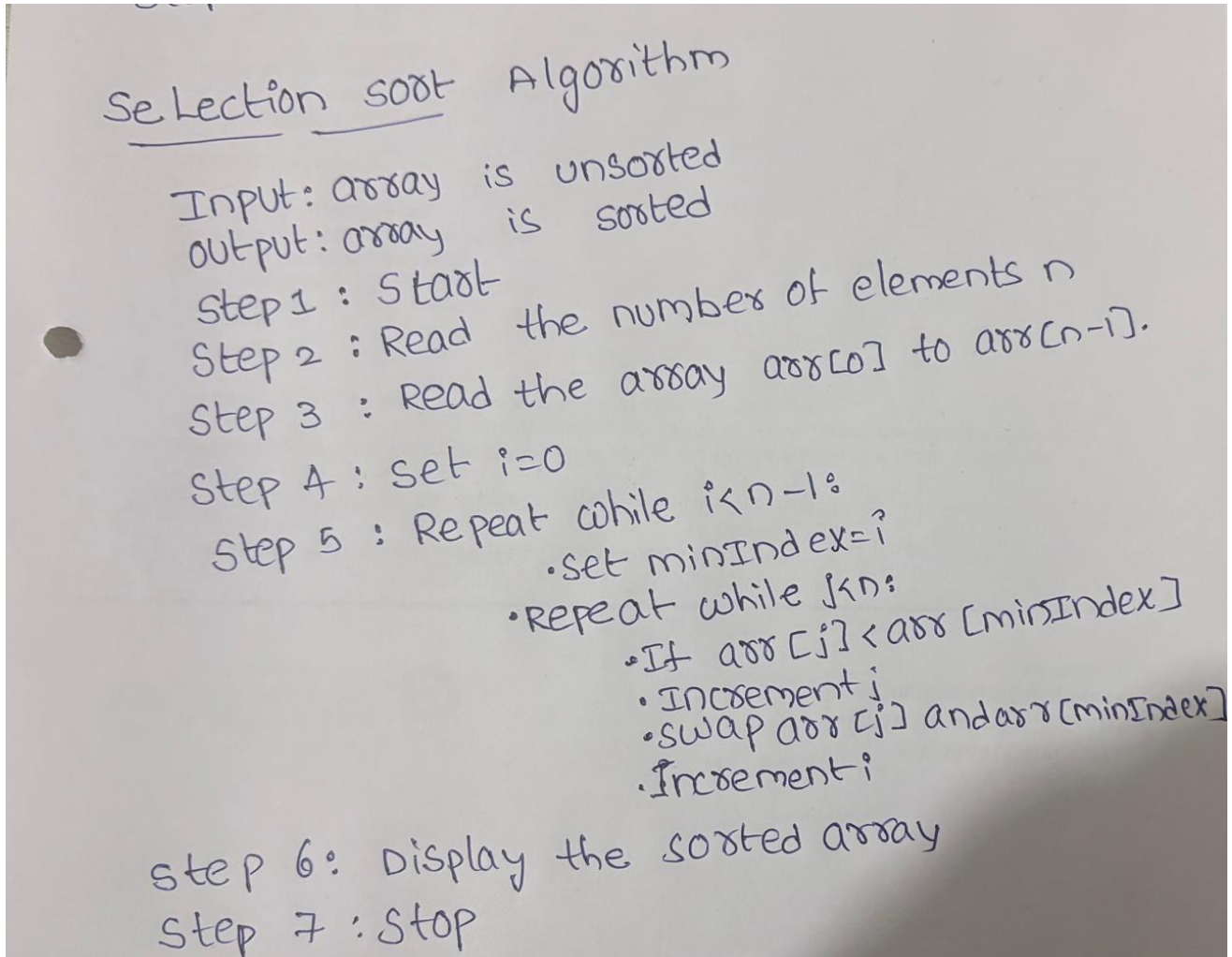
$$T(n) = n - 1$$

$$\text{Ignoring constant}$$

$$\boxed{T(n) = O(n)} \rightarrow \text{Best case}$$

2. Selection sort

Algo rthim:



Program:

```
#include <iostream>
```

```
using namespace std;
```

```
class selectionsort
```

```
{
```

```
public:
```

```
int size;

int arr[100];

int i;

void arrayinput()
{
    cout << "enter the size of array" << endl;
    cin >> size;
    cout << "enter the array elements" << endl;
    for (i = 0; i < size; i++)
    {
        cin >> arr[i];
    }
}

void printarray()
{
    for (i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void selectionlogic()
{
```



```
int i, j, minIndex, temp;
for (i = 0; i < size - 1; i++)
{
    minIndex = i;
    for (j = i + 1; j < size; j++)
    {
        if (arr[j] < arr[minIndex])
        {
            minIndex = j;
        }
    }
    temp = arr[i];
    arr[i] = arr[minIndex];
    arr[minIndex] = temp;
}
};
```

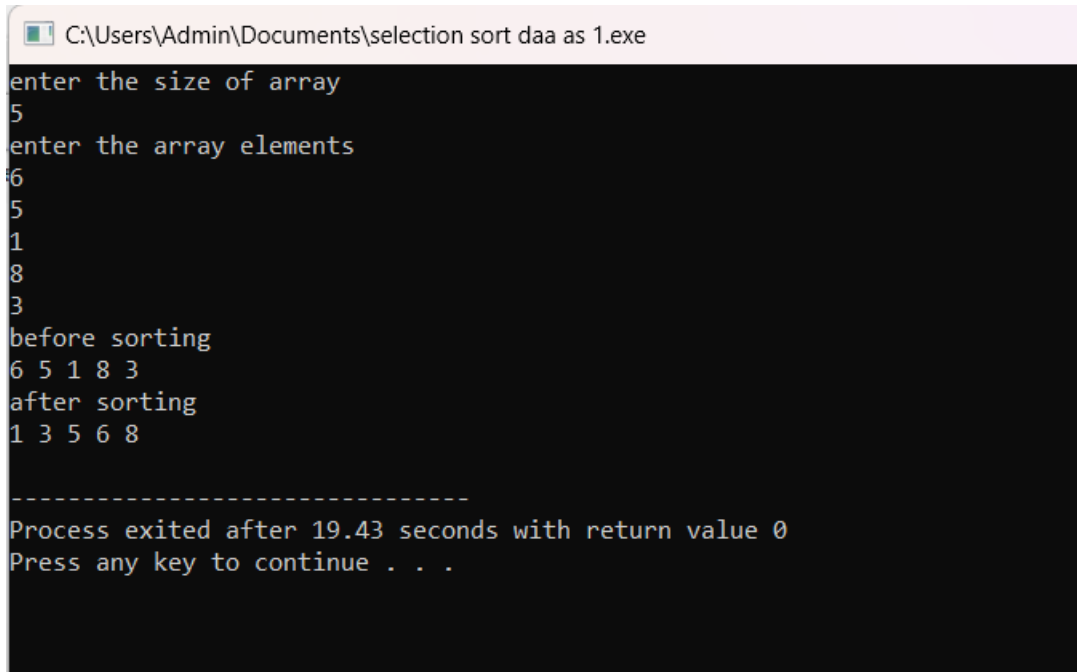
```
int main()
{
    selectionsort ss;
    ss.arrayinput();
    cout << "before sorting" << endl;
    ss.printarray();
    cout << "after sorting" << endl;
    ss.selectionlogic();
}
```

```
ss.printarray();
```

```
return 0;
```

```
}
```

Output:



```
C:\Users\Admin\Documents\selection sort daa as 1.exe
enter the size of array
5
enter the array elements
6
5
1
8
3
before sorting
6 5 1 8 3
after sorting
1 3 5 6 8

-----
Process exited after 19.43 seconds with return value 0
Press any key to continue . . .
```

Time complexity:

Selection sort

```
for (int i=0; i<n; i++) { —————  $2n+2$ 
    int min=i;
```

```
    for (int j=i+1; j<n; j++) { —————  $\frac{n(n-1)}{2}$ 
        if (arr[j] < arr[min]) { —————  $\frac{n(n-1)}{2}$ 
            min=j;
        }
```

```
    } if (min != i) { temp=arr[i];
        arr[i]=arr[min];
        arr[min]=temp; } —————  $\frac{n(n-1)}{2}$ 
    }
```

Worst case

The array is in unsorted

$$T(n) = 2n + 2 + \frac{3(n^2 - n)}{2}$$

$$= 2n + 2 + \frac{3n^2 - 3n}{2} \rightarrow \text{Ignoring constant}$$

$$= n^2 + n - 1$$

$$T(n) = O(n^2) \rightarrow \text{Worst case}$$

Average case

The array is in random order

$$T(n) = 2n + 2 + \frac{3(n^2 - n)}{2}$$

$$= 2n + 2 + \frac{3n^2 - 3n}{2} \rightarrow \text{Ignoring constant}$$

$$= n^2 + n - 1$$

$$T(n) = O(n^2) \rightarrow \text{Average case}$$

Best case

The array is sorted, but sort the unsorted part

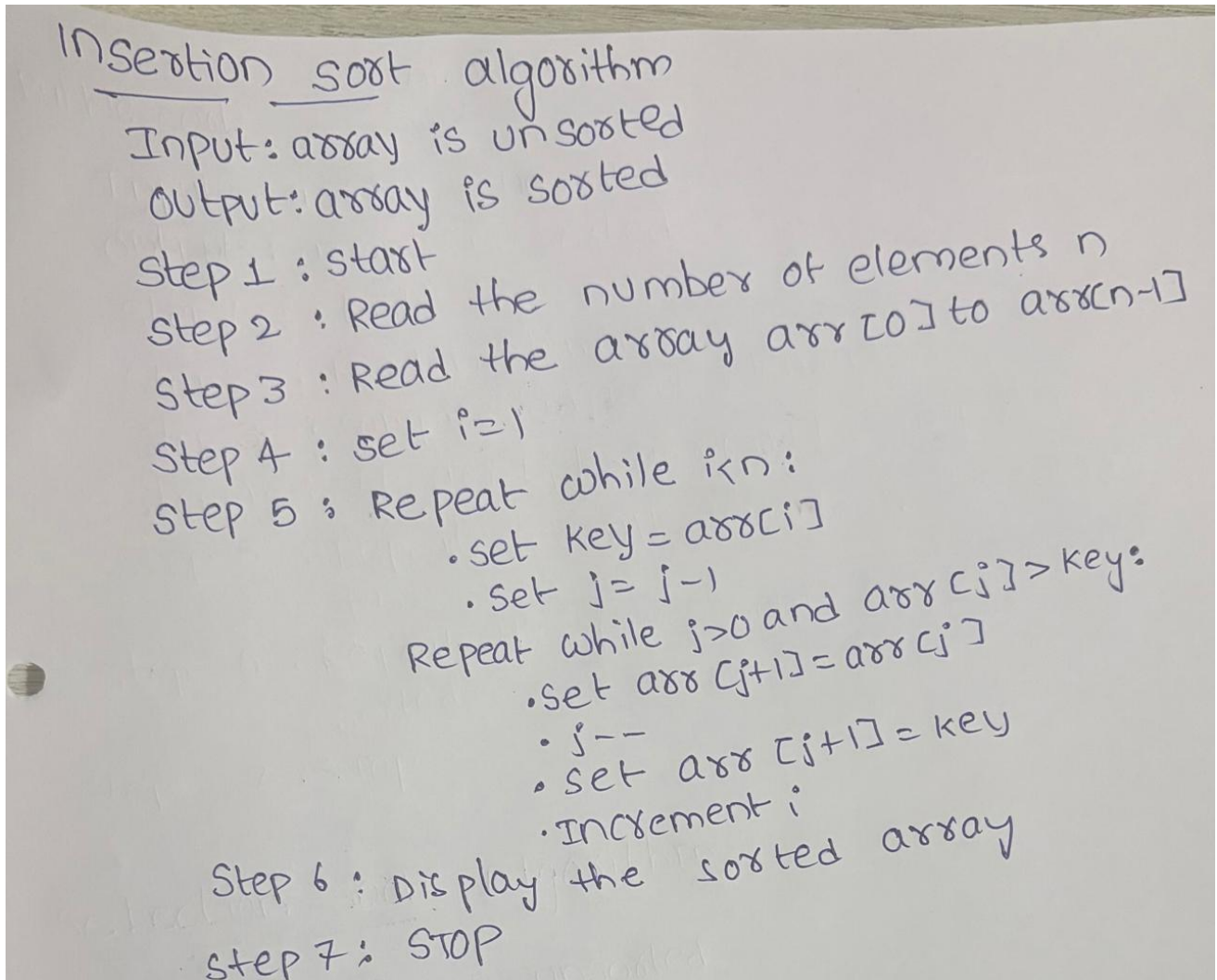
$$T(n) = 2n + 2 + \frac{3(n^2 - n)}{2}$$

$$= n^2 + n - 1$$

$$T(n) = O(n^2) \rightarrow \text{Best case}$$

3.Insertion sort

Algoarthim:



Insertion sort algorithm
Input: array is unsorted
Output: array is sorted
Step 1 : start
Step 2 : Read the number of elements n
Step 3 : Read the array arr[0] to arr[n-1]
Step 4 : set i=1
Step 5 : Repeat while i<n:
 • set key = arr[i]
 • set j = j-1
 Repeat while j>0 and arr[j]>key:
 • set arr[j+1] = arr[j]
 • j--
 • set arr[j+1] = key
 • Increment i
Step 6 : Display the sorted array
Step 7 : STOP

Program:

```
#include <iostream>
```

```
using namespace std;
```

```
class insertionsort
```

```
{  
  
public:  
  
    int size;  
  
    int arr[100];  
  
    int i;  
  
    void arrayinput()  
    {  
        cout << "enter the size of array" << endl;  
        cin >> size;  
        cout << "enter the array elements" << endl;  
        for (i = 0; i < size; i++)  
        {  
            cin >> arr[i];  
        }  
    }  
  
    void printarray()  
    {  
        for (i = 0; i < size; i++)  
        {  
            cout << arr[i] << " ";  
        }  
        cout << endl;
```

}

void insertionlogic()

{

int i, j, key;

for (i = 1; i < size; i++)

{

key = arr[i];

j = i - 1;

while (j >= 0 && arr[j] > key)

{

arr[j + 1] = arr[j];

j = j - 1;

}

arr[j + 1] = key;

}

}

};

int main()

{

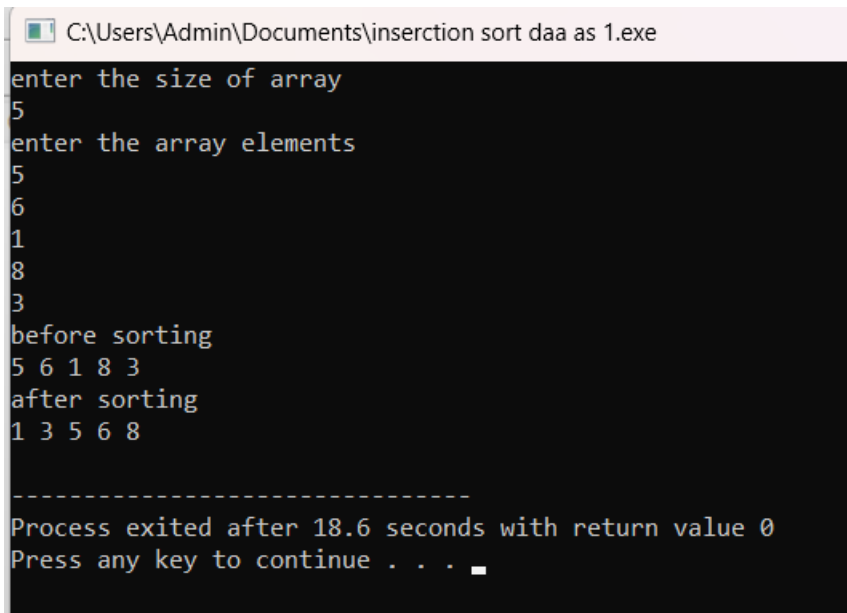
insertionsort is;

is.arrayinput();

cout << "before sorting" << endl;

```
is.printarray();  
cout << "after sorting" << endl;  
is.insertionlogic();  
is.printarray();  
return 0;  
}
```

Output:



```
C:\Users\Admin\Documents\inserction sort daa as 1.exe  
enter the size of array  
5  
enter the array elements  
5  
6  
1  
8  
3  
before sorting  
5 6 1 8 3  
after sorting  
1 3 5 6 8  
  
-----  
Process exited after 18.6 seconds with return value 0  
Press any key to continue . . .
```



Time complexity:

Insertion sort

```

for (i=1; i<n; i++) {  $\rightarrow n-1$ 
    temp = arr[i];  $\rightarrow n-1$ 
    j = i-1;  $\rightarrow n-1$ 
    while (j>0 && arr[j] > temp) {  $\rightarrow \frac{n(n-1)}{2}$ 
        arr[j+1] = arr[j];  $\rightarrow n-1$ 
        j--;  $\rightarrow n-1$ 
    }
    arr[j+1] = temp;  $\rightarrow n-1$ 
}

```

Worst case

The array is in unsorted

$$T(n) = (n-1) + (n-1) + (n-1) + \frac{n(n-1)}{2} + (n-1) + (n-1) + (n-1)$$

$$= 6n - 6 + \frac{n(n-1)}{2}$$

$$= 6n - 6 + \frac{n^2 - n}{2} \rightarrow \text{Ignoring constant}$$

$$= \cancel{6n} + n^2 - \cancel{6}$$

$$= \frac{n^2}{2} \rightarrow \text{Worst case}$$

$$T(n) = O(n^2)$$

Best case

The array is already sorted

$$T(n) = (n-1) \rightarrow \text{Ignoring constant}$$

$$T(n) = O(n) \rightarrow \text{Best case}$$

Average case

The array is in random

$$T(n) = (n-1) + (n-1) + \frac{n(n-1)}{2} \rightarrow \text{Ignoring constant}$$

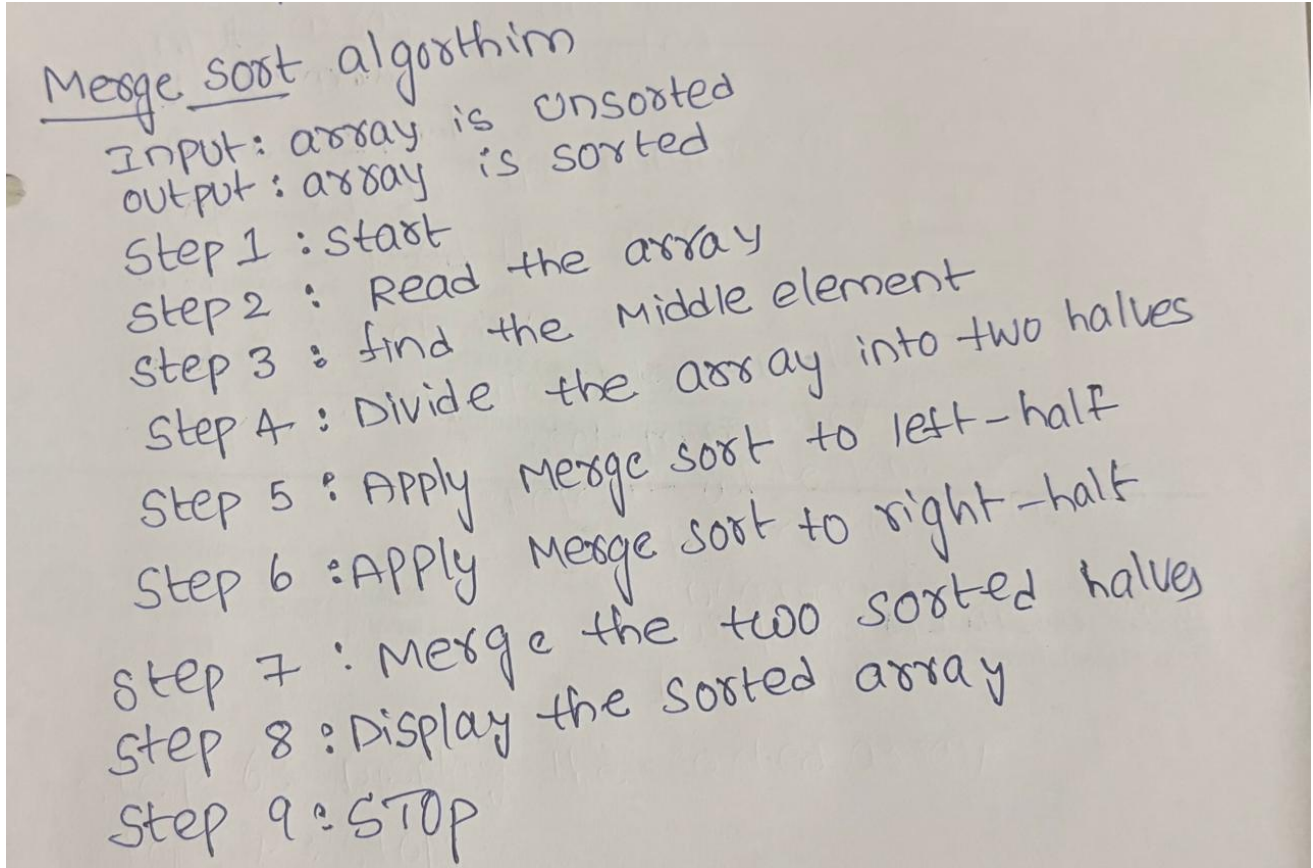
$$= 2n + \frac{n^2 - n}{2}$$

$$= \cancel{2n} + n^2 - \cancel{n}$$

$$T(n) = O(n^2) \rightarrow \text{Average case}$$

4. Merge sort

Algorithm:



Program:

```
#include <iostream>
```

```
using namespace std;
```

```
class mergesort
```

```
{
```

```
public:
```

```
    int size;
```

```
    int arr[100];
```

```
int i;
```

```
void arrayinput()
```

```
{  
    cout << "enter the size of array" << endl;  
    cin >> size;  
    cout << "enter the array elements" << endl;  
    for (i = 0; i < size; i++)  
    {  
        cin >> arr[i];  
    }  
}
```

```
void printarray()
```

```
{  
    for (i = 0; i < size; i++)  
    {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}
```

```
void merge(int low, int mid, int high)
```

```
{  
    int temp[100];  
    int i = low;
```



```
int j = mid + 1;
```

```
int k = low;
```

```
while (i <= mid && j <= high)
```

```
{
```

```
    if (arr[i] <= arr[j])
```

```
    {
```

```
        temp[k] = arr[i];
```

```
        i++;
```

```
    }
```

```
    else
```

```
    {
```

```
        temp[k] = arr[j];
```

```
        j++;
```

```
    }
```

```
    k++;
```

```
}
```

```
while (i <= mid)
```

```
{
```

```
    temp[k] = arr[i];
```

```
    i++;
```

```
    k++;
```

```
}
```

```
while (j <= high)
```



```
{  
    temp[k] = arr[j];  
    j++;  
    k++;  
}
```

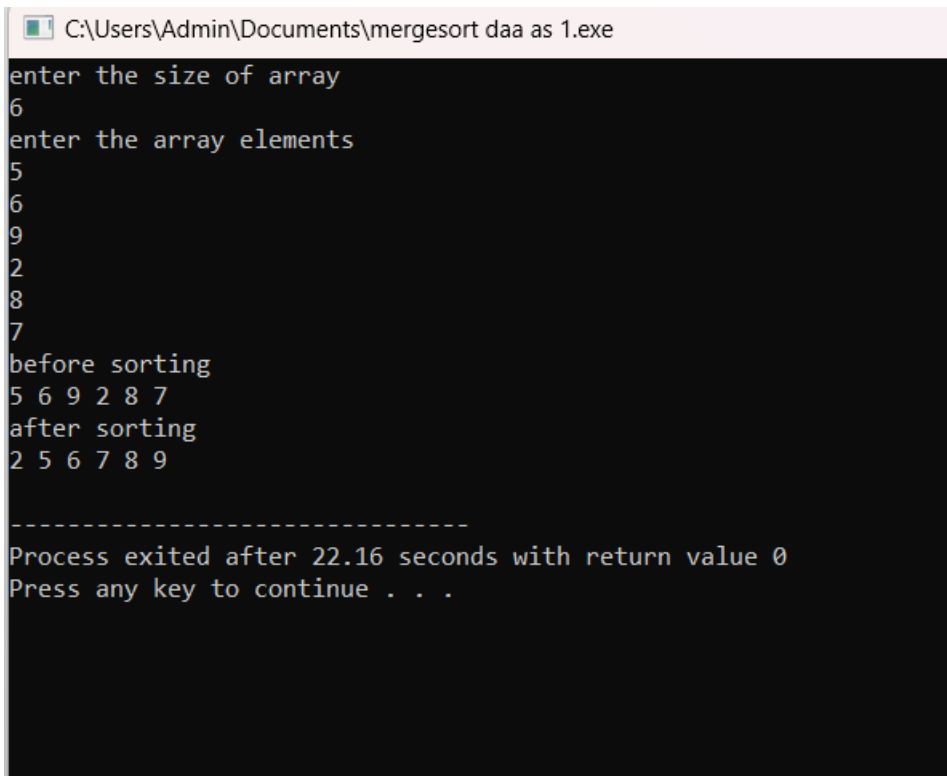
```
for (i = low; i <= high; i++)  
{  
    arr[i] = temp[i];  
}  
}
```

```
void mergellogic(int low, int high)  
{  
    if (low < high)  
    {  
        int mid = (low + high) / 2;  
        mergellogic(low, mid);  
        mergellogic(mid + 1, high);  
        merge(low, mid, high);  
    }  
}  
};
```

```
int main()  
{
```

```
mergesort ms;  
ms.arrayinput();  
cout << "before sorting" << endl;  
ms.printarray();  
cout << "after sorting" << endl;  
ms.mergellogic(0, ms.size - 1);  
ms.printarray();  
return 0;  
}
```

Output:



```
C:\Users\Admin\Documents\mergesort daa as 1.exe  
enter the size of array  
6  
enter the array elements  
5  
6  
9  
2  
8  
7  
before sorting  
5 6 9 2 8 7  
after sorting  
2 5 6 7 8 9  
  
-----  
Process exited after 22.16 seconds with return value 0  
Press any key to continue . . .
```

Time complexity:

Merge sort

```
void MergeLogic (int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        MergeLogic (low, mid);
        MergeLogic (mid + 1, high);
        MergeLogic (low, mid, high);
    }
}
```

Worst case

The array is unsorted
 $T(n) = 2T(n/2) + O(n)$
 using Master Method

$a=2, b=2, K=1, P=0$

$\log_a b = K$

$\log_2 2 = 1$

$1=1$

case 2 $P > -1$

$T(n) = O(n^K (\log^n n))$

$= (n * (\log n))$

$T(n) = O(n \log n)$ → worst case

Average case

the array is random
 $T(n) = 2T(n/2) + O(n)$

$T(n) = aT(n/b) + f(n)$

$f(n) = n^K * (\log^P n)$

$K=1, P=0, a=2, b=2$

$\log_a b = K \Rightarrow \log_2 2 = 1$

case 2 $P > -1$

$T(n) = O(n^K * (\log^{P+1} n))$

$= O(n * (\log n))$

$T(n) = O(n \log n)$ Average case

Best case

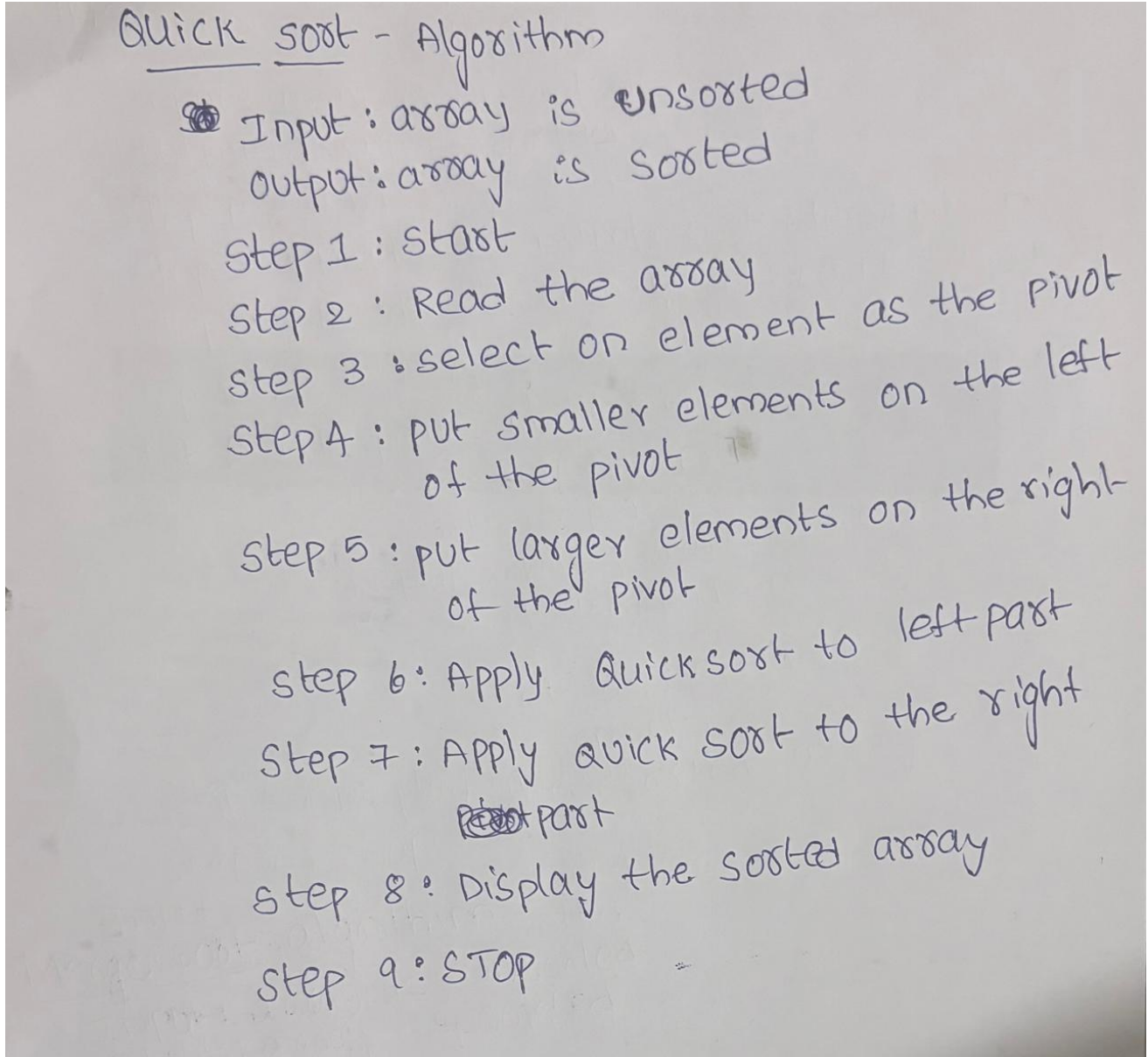
The array is already sorted
 so The divide and conquer then array became $n/2$

$T(n) = 2T(n/2) + O(n)$

$T(n) = O(n \log n)$ → Best case

5.Quick sort

Algorithm:



Program:

```
# <iostream>include  
using namespace std;
```

```
class quicksort
```

```
{  
  
public:  
  
    int size;  
  
    int arr[100];  
  
    int i;  
  
    void arrayinput()  
    {  
        cout << "enter the size of array" << endl;  
        cin >> size;  
        cout << "enter the array elements" << endl;  
        for (i = 0; i < size; i++)  
        {  
            cin >> arr[i];  
        }  
    }  
  
    void printarray()  
    {  
        for (i = 0; i < size; i++)  
        {  
            cout << arr[i] << " ";  
        }  
        cout << endl;  
    }  
}
```



```
int partition(int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    int temp;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quicklogic(int low, int high)
{
```

```
    if (low < high)
    {
        int pi = partition(low, high);
        quicklogic(low, pi - 1);
        quicklogic(pi + 1, high);
    }
}

};

int main()
{
    quicksort qs;
    qs.arrayinput();
    cout << "before sorting" << endl;
    qs.printarray();
    cout << "after sorting" << endl;
    qs.quicklogic(0, qs.size - 1);
    qs.printarray();
    return 0;
}
```

Output:

```
C:\Users\Admin\Documents\quicksort daa as 1.exe
enter the size of array
6
enter the array elements
12
5
9
11
2
9
before sorting
12 5 9 11 2 9
after sorting
2 5 9 9 11 12

-----
Process exited after 21.53 seconds with return value 0
Press any key to continue . . .
```

Time complexity:

Quick Sort

```
void quick (int low, int high) {
    if (low < high) {
        int pi = partition (low, high);
        quicklog (low, pi-1);
        quicklog (pi+1, high);
    }
}
```

Best case
occurs, The array divided in two halves the best case

$$T(n) = 2T(n/2) + O(n)$$

$$T(n) = n^k \times (\log^p n)$$

$$a=2, b=2, k=1, p=0$$

$$\log_b a = k$$

$$\log_2 2 = k$$

$$1 = k$$

case 2 $p > -1$

$$T(n) = O(n^k \times \log^p n)$$

$$= O(n^1 \times \log^1 n)$$

$$T(n) = O(n \log n)$$

Best case
Average case

Worst case
The array The pivot became the smallest or largest
selected
Recursive.

$$T(n) = (n-1) + O(n)$$

$$T(n) = (n-2) + (n-1) + n$$

$$T(n) = (n-3) + (n-2) + (n-1) + n$$

$$T(n) = 1 + 2 + 3 + \dots + n$$

$$1 + 2 + 3 + \dots + n = \frac{n(n-1)}{2}$$

$$T(n) = O(n^2) \text{ - worst case}$$

Conclusion:

In this practical, we implemented and studied five sorting algorithms: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. We learned how each algorithm works and compared their time complexities.

Bubble Sort, Selection Sort, and Insertion Sort are simple but can be slow for large data. Merge Sort and Quick Sort are generally faster and more suitable for large datasets. Therefore, the choice of sorting algorithm depends on the size and type of data.